

```
In [4]: import os

print("Current Working Directory:")
print(os.getcwd())

print("\nFiles and folders in this directory:")
print(os.listdir())
```

Current Working Directory:
D:\CustomerSegmentation

Files and folders in this directory:
['.ipynb_checkpoints', 'data', 'Untitled.ipynb', 'venv']

```
In [5]: df = pd.read_csv("data/ecommerce_customer_data_large.csv")
```

```
In [6]: import os
print(os.getcwd())
print(os.listdir())
```

D:\CustomerSegmentation
['.ipynb_checkpoints', 'data', 'Untitled.ipynb', 'venv']

```
In [7]: import os

print(os.listdir("data"))
```

['.ipynb_checkpoints', 'ecommerce_customer_data_large.csv']

```
In [8]: import pandas as pd

df = pd.read_csv("data/ecommerce_customer_data_large.csv")
df.head()
```

Out[8]:

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Age
0	44605	2023-05-03 21:30:02	Home	177	1	2427	PayPal	31
1	44605	2021-05-16 13:57:44	Electronics	174	3	2448	PayPal	31
2	44605	2020-07-13 06:16:57	Books	413	1	2345	Credit Card	31
3	44605	2023-01-17 13:14:36	Electronics	396	3	937	Cash	31
4	44605	2021-05-01 11:29:27	Books	259	4	2598	PayPal	31

In [9]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID           250000 non-null  int64
1   Purchase Date         250000 non-null  object
2   Product Category      250000 non-null  object
3   Product Price         250000 non-null  int64
4   Quantity              250000 non-null  int64
5   Total Purchase Amount 250000 non-null  int64
6   Payment Method        250000 non-null  object
7   Customer Age          250000 non-null  int64
8   Returns               202618 non-null  float64
9   Customer Name         250000 non-null  object
10  Age                   250000 non-null  int64
11  Gender                250000 non-null  object
12  Churn                 250000 non-null  int64
dtypes: float64(1), int64(7), object(5)
memory usage: 24.8+ MB
```

In [10]: `print("Rows and Columns:", df.shape)`

Rows and Columns: (250000, 13)

In [11]: `print(df.columns)`

```
Index(['Customer ID', 'Purchase Date', 'Product Category', 'Product Price',
      'Quantity', 'Total Purchase Amount', 'Payment Method', 'Customer Age',
      'Returns', 'Customer Name', 'Age', 'Gender', 'Churn'],
      dtype='object')
```

In [12]: `df.isnull().sum()`

```
Out[12]: Customer ID           0
Purchase Date           0
Product Category        0
Product Price           0
Quantity                0
Total Purchase Amount    0
Payment Method          0
Customer Age            0
Returns                 47382
Customer Name           0
Age                     0
Gender                  0
Churn                   0
dtype: int64
```

In [13]: `df.dropna(inplace=True)`

```
print(df.shape)
```

(202618, 13)

In [14]: `print("Duplicate Rows:", df.duplicated().sum())`

Duplicate Rows: 0

```
In [15]: df.drop_duplicates(inplace=True)
```

```
print(df.shape)
```

```
(202618, 13)
```

```
In [16]: df["Purchase Date"] = pd.to_datetime(df["Purchase Date"])
```

```
In [17]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 202618 entries, 0 to 249999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID           202618 non-null int64
1   Purchase Date         202618 non-null datetime64[ns]
2   Product Category     202618 non-null object
3   Product Price         202618 non-null int64
4   Quantity              202618 non-null int64
5   Total Purchase Amount 202618 non-null int64
6   Payment Method        202618 non-null object
7   Customer Age          202618 non-null int64
8   Returns               202618 non-null float64
9   Customer Name         202618 non-null object
10  Age                   202618 non-null int64
11  Gender                202618 non-null object
12  Churn                 202618 non-null int64
dtypes: datetime64[ns](1), float64(1), int64(7), object(4)
memory usage: 21.6+ MB
```

```
In [18]: df["Year"] = df["Purchase Date"].dt.year
```

```
In [19]: df["Month"] = df["Purchase Date"].dt.month
```

```
In [20]: df["Month Name"] = df["Purchase Date"].dt.month_name()
```

```
In [21]: df["Day"] = df["Purchase Date"].dt.day
```

```
In [22]: df["Weekday"] = df["Purchase Date"].dt.day_name()
```

```
In [23]: df["Quarter"] = df["Purchase Date"].dt.quarter
```

```
In [24]: df.head()
```

Out[24]:

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Age
0	44605	2023-05-03 21:30:02	Home	177	1	2427	PayPal	31
1	44605	2021-05-16 13:57:44	Electronics	174	3	2448	PayPal	31
2	44605	2020-07-13 06:16:57	Books	413	1	2345	Credit Card	31
3	44605	2023-01-17 13:14:36	Electronics	396	3	937	Cash	31
4	44605	2021-05-01 11:29:27	Books	259	4	2598	PayPal	31

In [25]: df.describe()

Out[25]:

	Customer ID	Purchase Date	Product Price	Quantity	Total Purchase Amount
count	202618.000000	202618	202618.000000	202618.000000	202618.000000
mean	25020.140234	2021-11-07 02:54:34.356700672	254.899066	3.008420	2725.817662
min	1.000000	2020-01-01 00:07:26	10.000000	1.000000	100.000000
25%	12599.250000	2020-12-03 12:46:27.249999872	133.000000	2.000000	1478.000000
50%	25018.000000	2021-11-06 19:26:45.500000	255.000000	3.000000	2727.000000
75%	37444.000000	2022-10-11 08:24:34	377.000000	4.000000	3975.000000
max	50000.000000	2023-09-13 18:42:49	500.000000	5.000000	5350.000000
std	14412.388674	NaN	141.720425	1.414339	1442.268491

In [26]: df.to_csv("data/cleaned_data.csv", index=False)

Data Cleaning & Feature Engineering

Data Cleaning

- Checked data types
- Removed missing values
- Removed duplicate records
- Converted Purchase Date to datetime format

Feature Engineering

Created the following features:

- Year
- Month
- Month Name
- Day
- Weekday
- Quarter

These features will help analyze customer purchasing behavior over time.

Exploratory Data Analysis (EDA)

The purpose of EDA is to understand customer purchasing behavior, identify trends, and visualize important patterns in the dataset.

```
In [27]: print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])

print("\nColumns:")
print(df.columns.tolist())
```

Number of Rows: 202618

Number of Columns: 19

Columns:

```
['Customer ID', 'Purchase Date', 'Product Category', 'Product Price', 'Quantity',
'Total Purchase Amount', 'Payment Method', 'Customer Age', 'Returns', 'Customer Name', 'Age', 'Gender', 'Churn', 'Year', 'Month', 'Month Name', 'Day', 'Weekday', 'Quarter']
```

```
In [28]: category_count = df["Product Category"].value_counts()

print(category_count)
```

Product Category

Clothing 50754

Home 50741

Electronics 50704

Books 50419

Name: count, dtype: int64

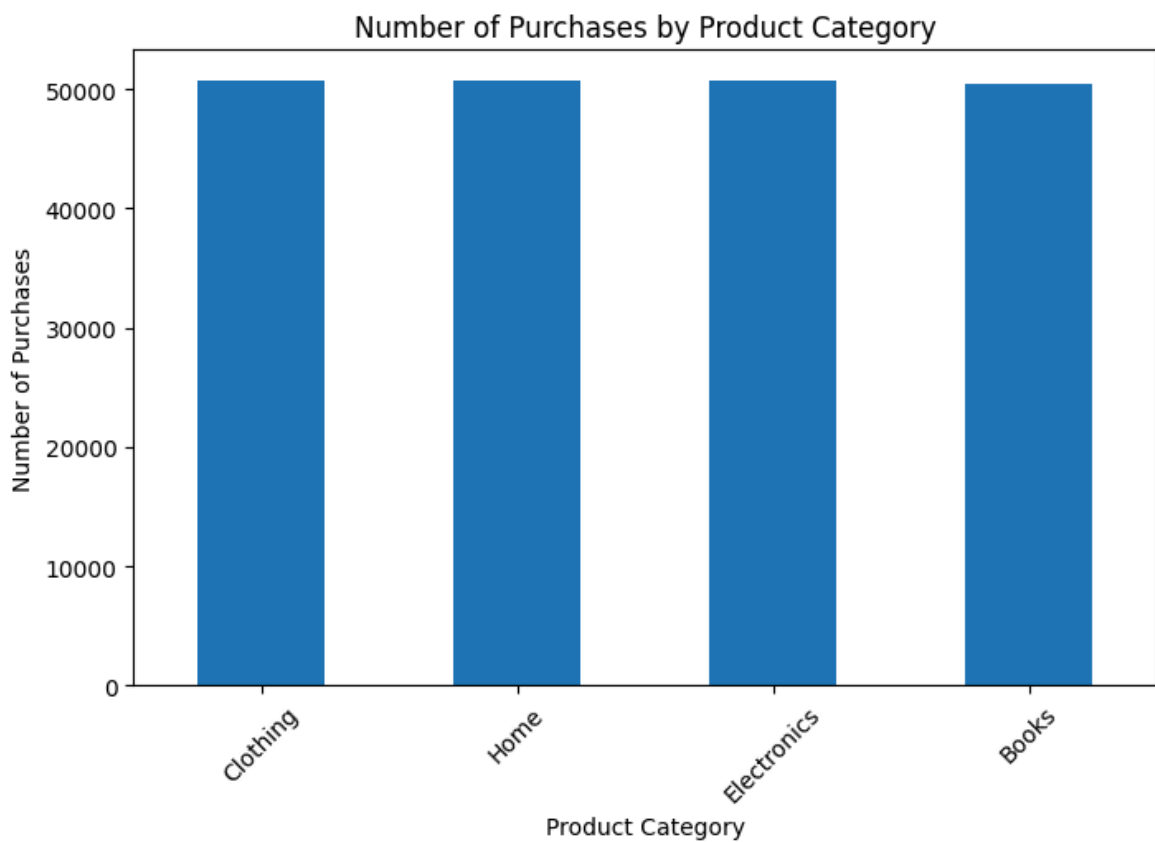
```
In [29]: category_count.plot(kind="bar", figsize=(8,5))

plt.title("Number of Purchases by Product Category")
```

```
plt.xlabel("Product Category")
plt.ylabel("Number of Purchases")

plt.xticks(rotation=45)

plt.show()
```



Observation

This chart shows which product categories are purchased most frequently.

```
In [30]: sales_category = df.groupby("Product Category")["Total Purchase Amount"].sum()

print(sales_category)
```

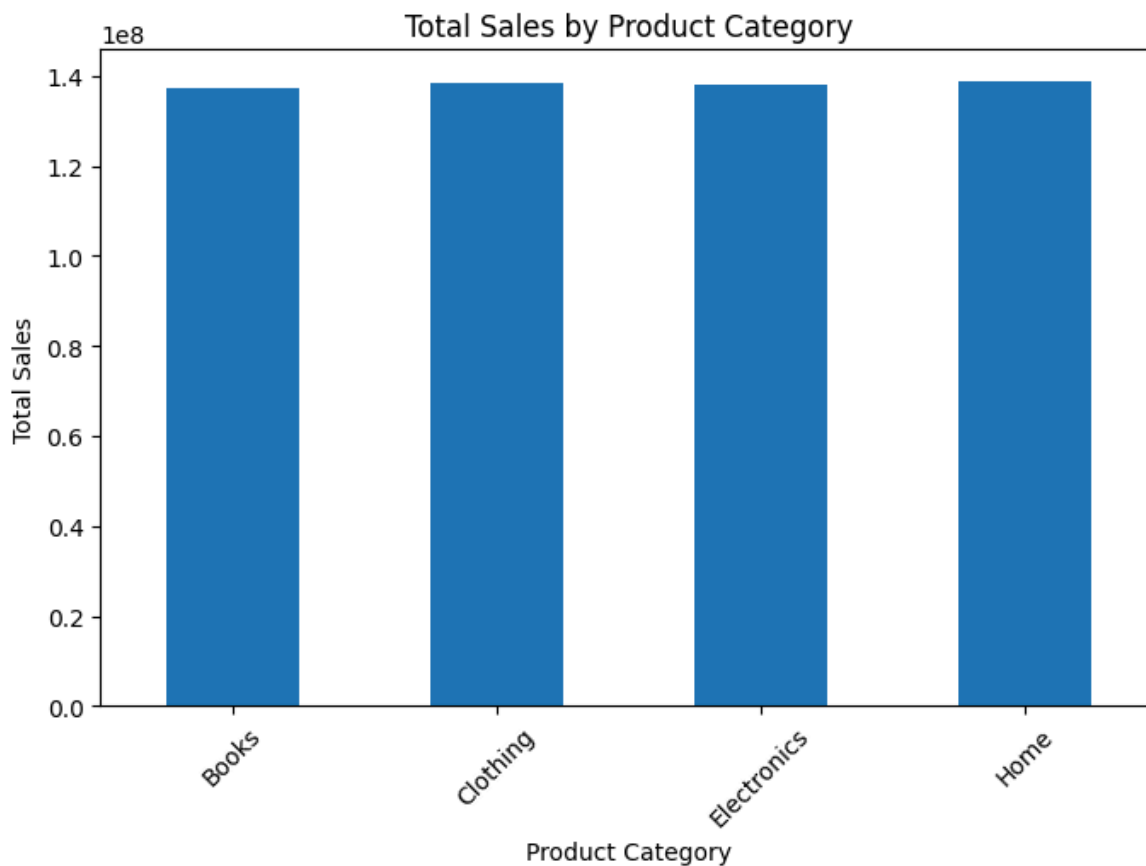
```
Product Category
Books          137145086
Clothing       138363540
Electronics    137884886
Home           138906211
Name: Total Purchase Amount, dtype: int64
```

```
In [31]: sales_category.plot(kind="bar", figsize=(8,5))

plt.title("Total Sales by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Sales")

plt.xticks(rotation=45)

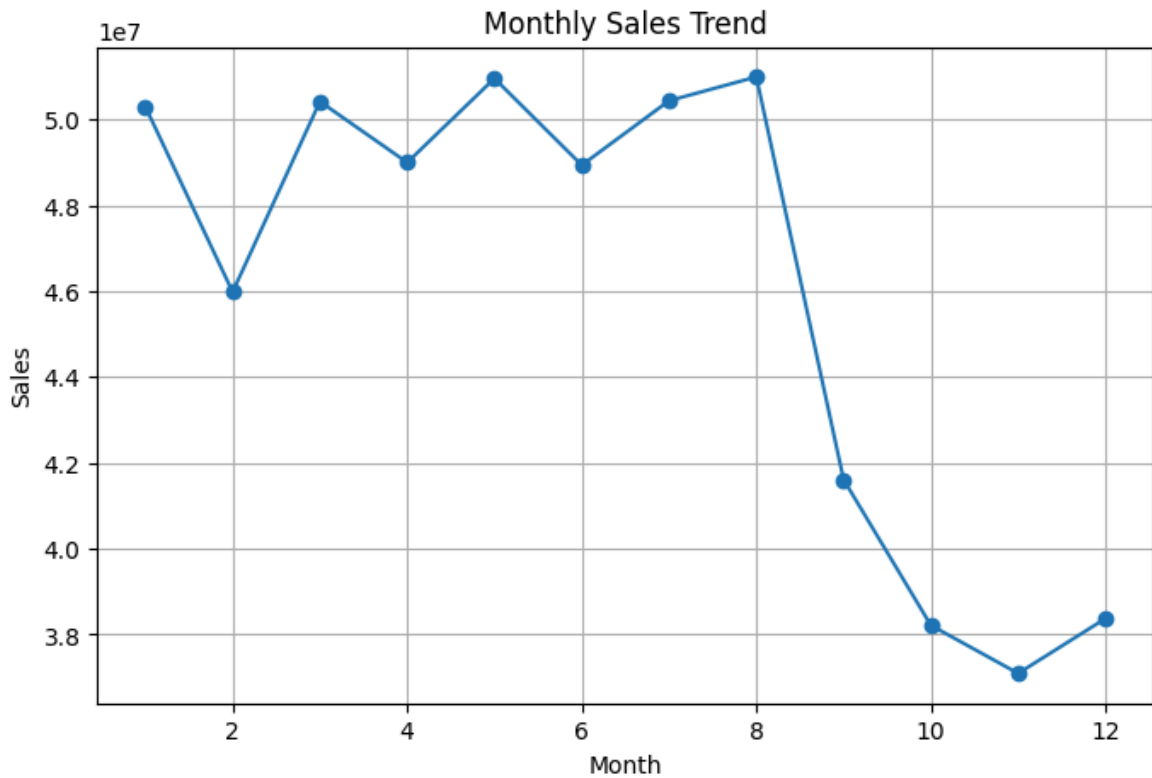
plt.show()
```



```
In [32]: monthly_sales = df.groupby("Month")["Total Purchase Amount"].sum()  
  
monthly_sales
```

```
Out[32]: Month  
1      50295819  
2      45980491  
3      50417750  
4      48996901  
5      50955398  
6      48934737  
7      50436195  
8      50991460  
9      41609578  
10     38214132  
11     37094782  
12     38372480  
Name: Total Purchase Amount, dtype: int64
```

```
In [33]: monthly_sales.plot(marker="o", figsize=(8,5))  
  
plt.title("Monthly Sales Trend")  
plt.xlabel("Month")  
plt.ylabel("Sales")  
  
plt.grid(True)  
  
plt.show()
```



Observation

This chart identifies seasonal sales trends and peak purchasing months.

```
In [34]: payment = df["Payment Method"].value_counts()
```

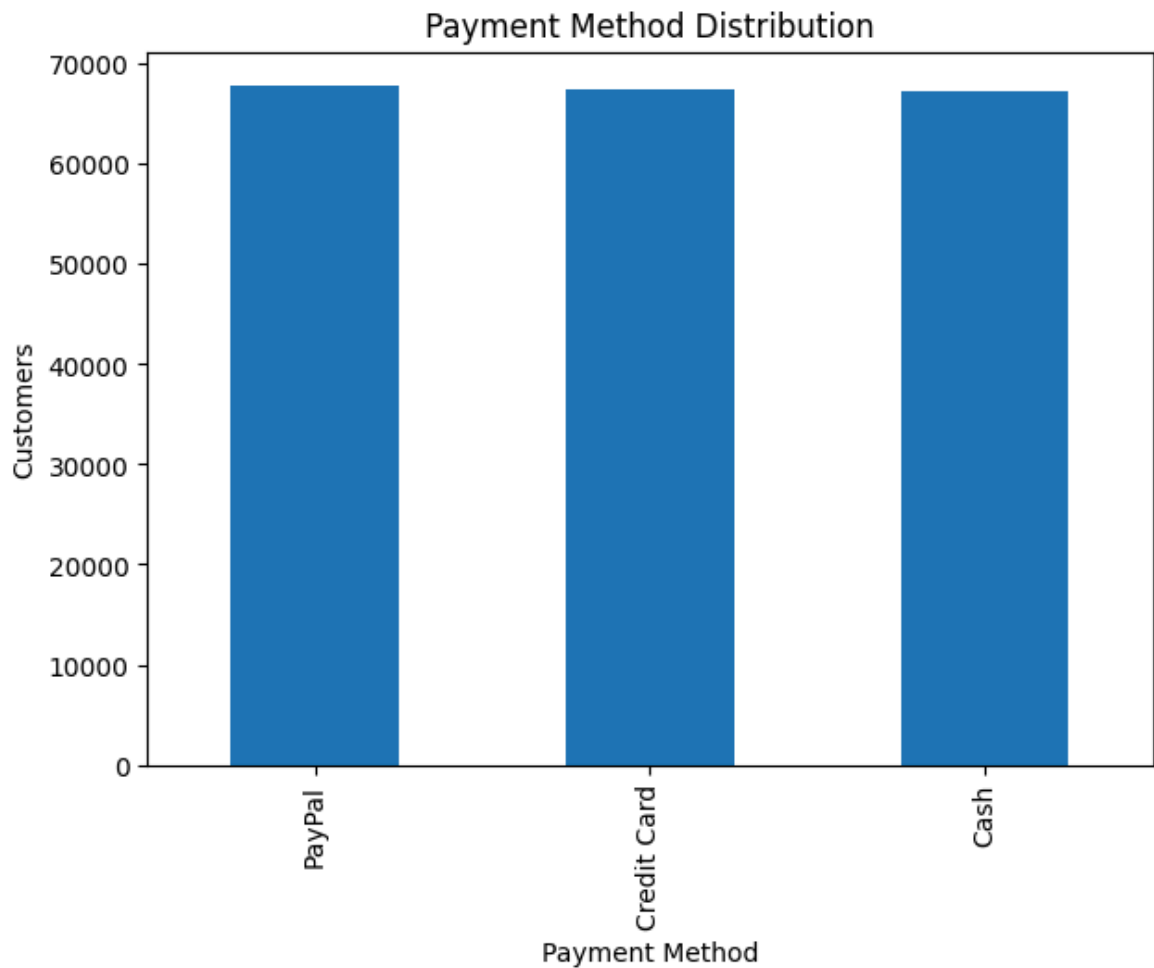
```
payment
```

```
Out[34]: Payment Method
PayPal      67811
Credit Card 67517
Cash        67290
Name: count, dtype: int64
```

```
In [35]: payment.plot(kind="bar", figsize=(7,5))
```

```
plt.title("Payment Method Distribution")
plt.xlabel("Payment Method")
plt.ylabel("Customers")

plt.show()
```

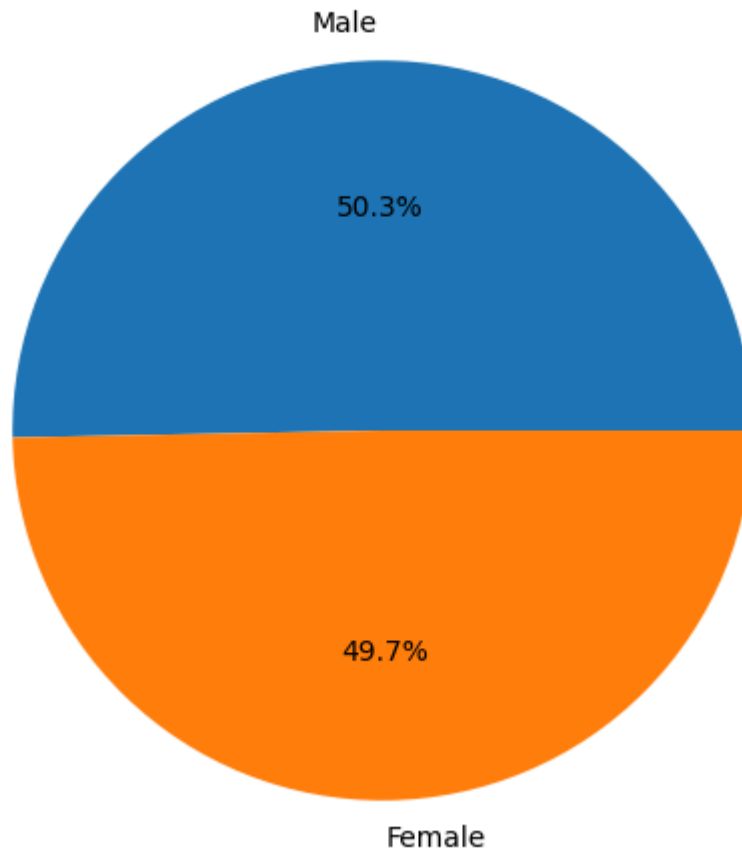



```
In [36]: gender = df["Gender"].value_counts()  
  
gender
```

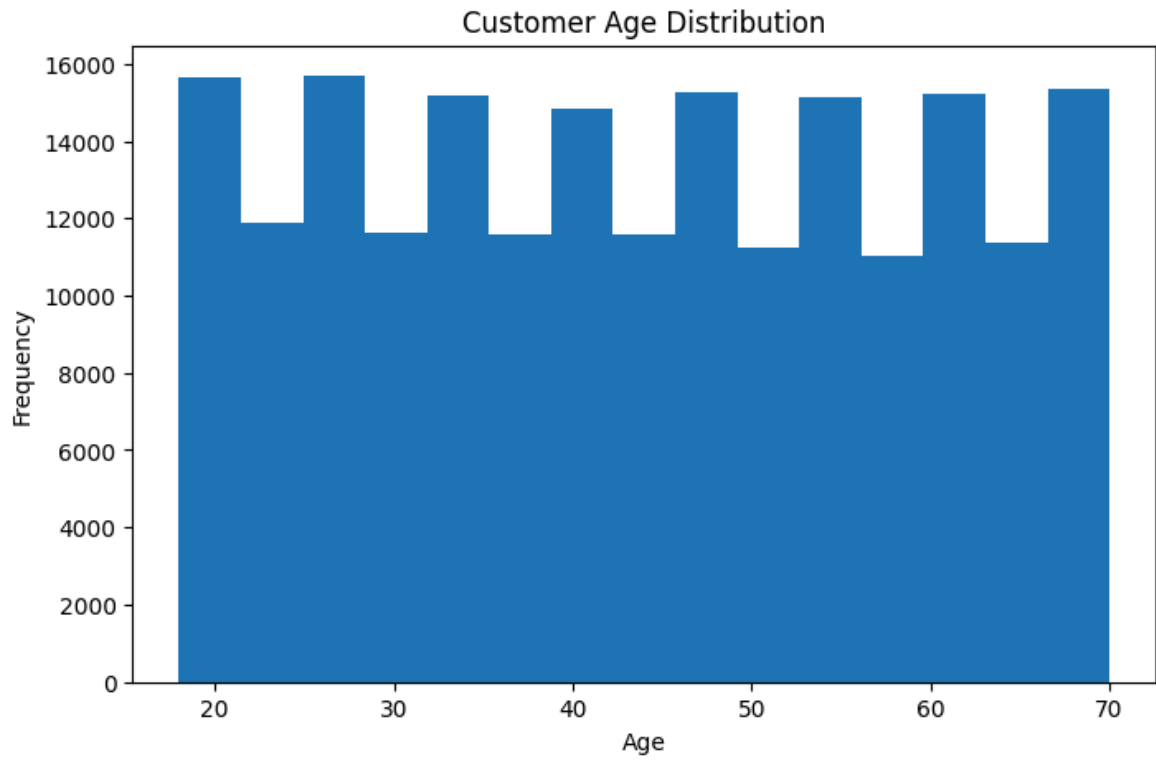
```
Out[36]: Gender  
Male      101919  
Female    100699  
Name: count, dtype: int64
```

```
In [37]: gender.plot(kind="pie",  
                    autopct="%1.1f%%",  
                    figsize=(6,6))  
  
plt.title("Gender Distribution")  
plt.ylabel("")  
  
plt.show()
```

Gender Distribution



```
In [38]: plt.figure(figsize=(8,5))  
plt.hist(df["Age"], bins=15)  
plt.title("Customer Age Distribution")  
plt.xlabel("Age")  
plt.ylabel("Frequency")  
plt.show()
```



```
In [39]: returns = df["Returns"].value_counts()
```

```
returns
```

```
Out[39]: Returns
1.0      101476
0.0      101142
Name: count, dtype: int64
```

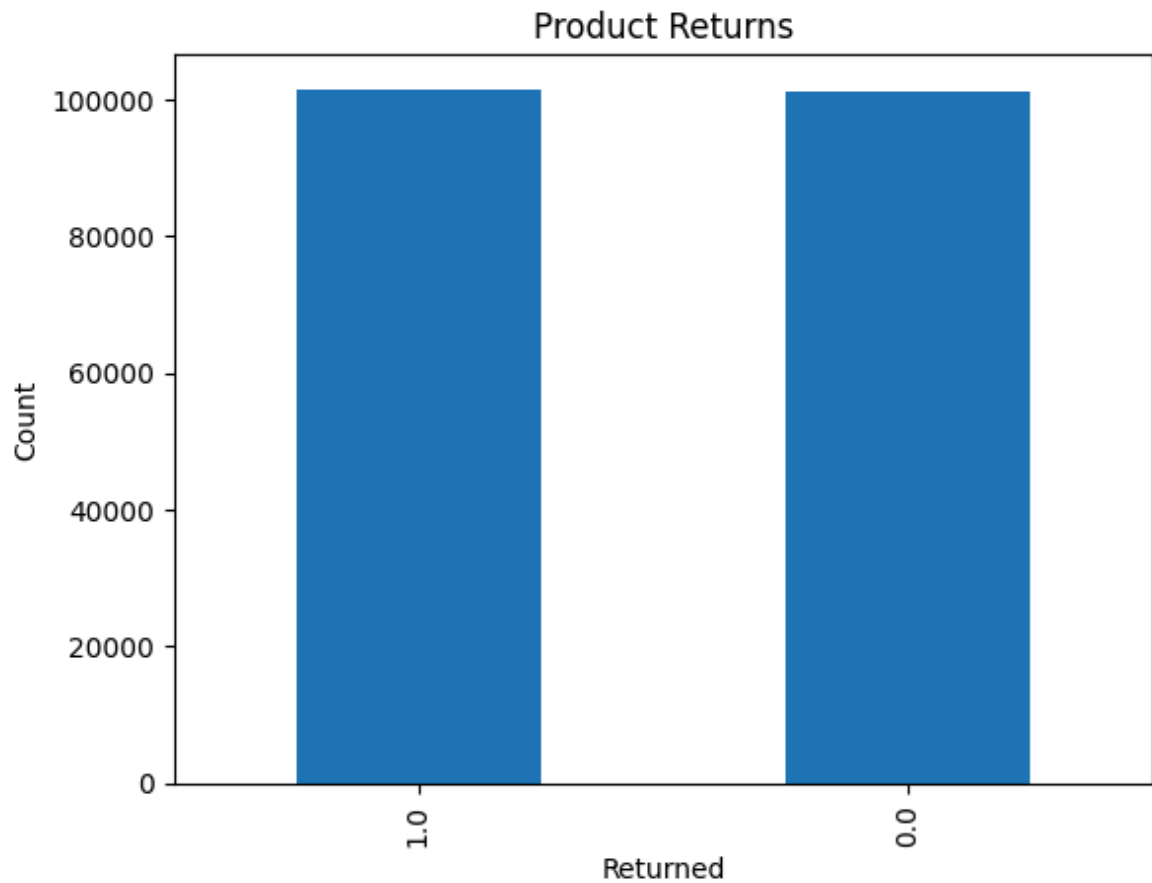
```
In [40]: returns.plot(kind="bar")

plt.title("Product Returns")

plt.xlabel("Returned")

plt.ylabel("Count")

plt.show()
```



```
In [41]: churn = df["Churn"].value_counts()
```

```
churn
```

```
Out[41]: Churn
0      161874
1       40744
Name: count, dtype: int64
```

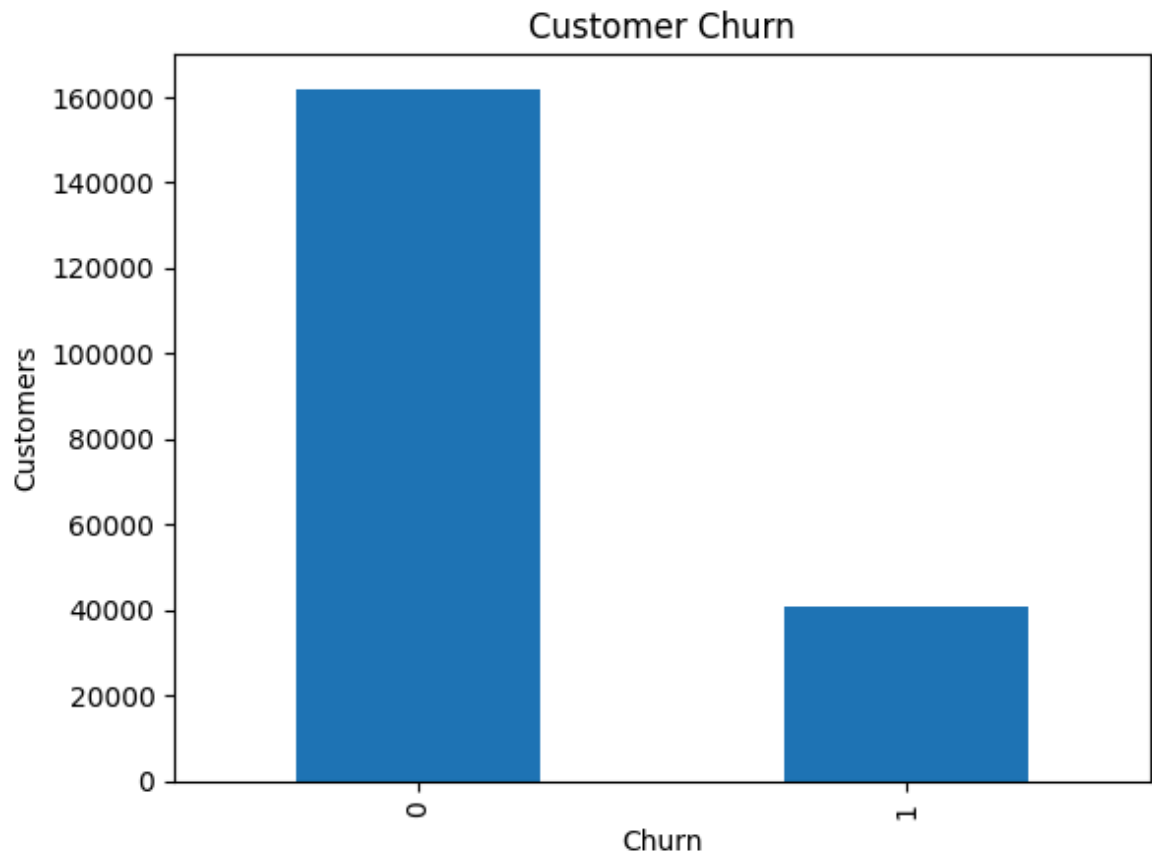
```
In [42]: churn.plot(kind="bar")

plt.title("Customer Churn")

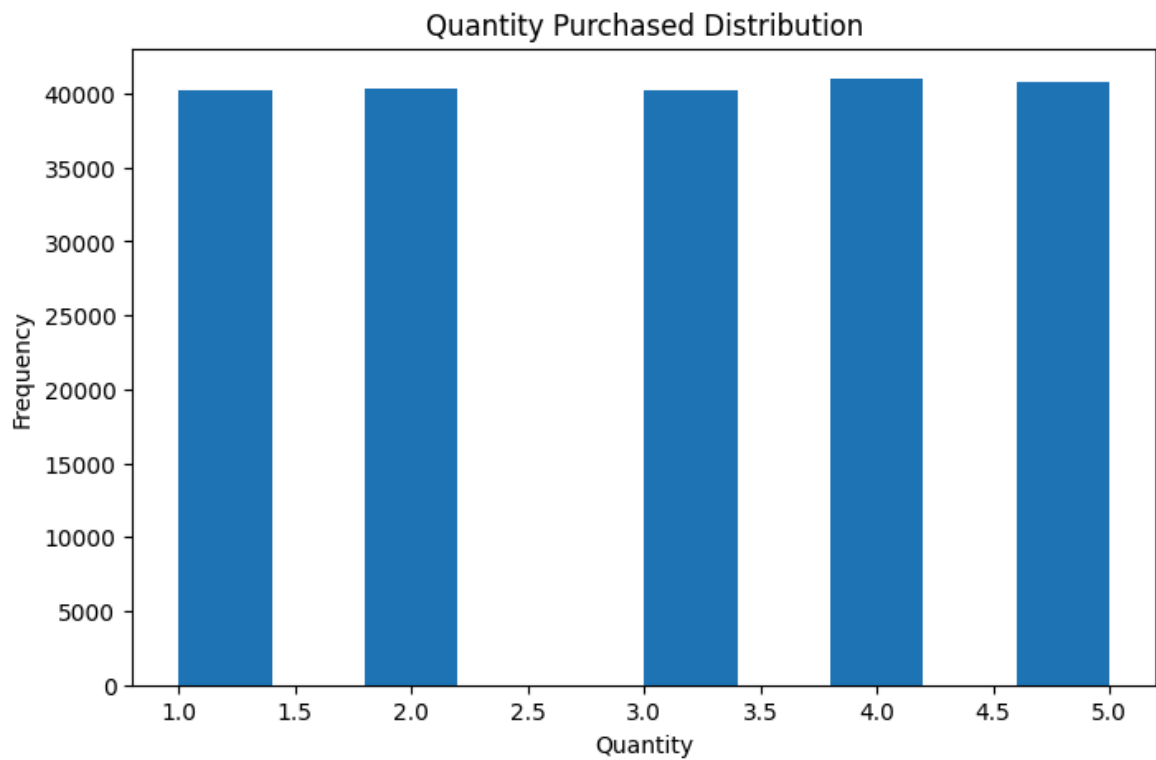
plt.xlabel("Churn")

plt.ylabel("Customers")

plt.show()
```



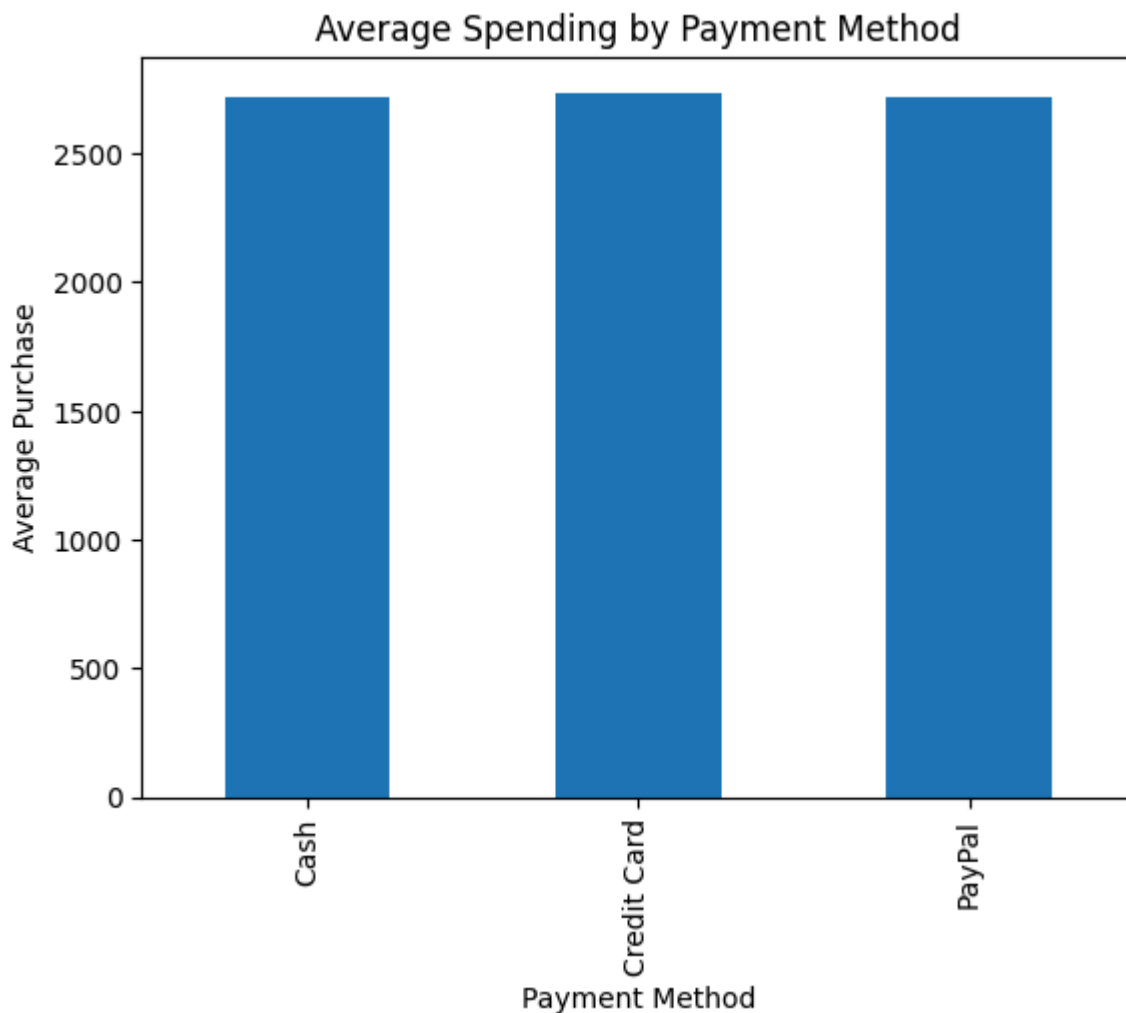
```
In [44]: plt.figure(figsize=(8,5))  
plt.hist(df["Quantity"], bins=10)  
plt.title("Quantity Purchased Distribution")  
plt.xlabel("Quantity")  
plt.ylabel("Frequency")  
plt.show()
```



```
In [45]: avg_payment = df.groupby("Payment Method")["Total Purchase Amount"].mean()  
  
avg_payment
```

```
Out[45]: Payment Method  
Cash      2718.298603  
Credit Card  2737.087208  
PayPal      2722.058265  
Name: Total Purchase Amount, dtype: float64
```

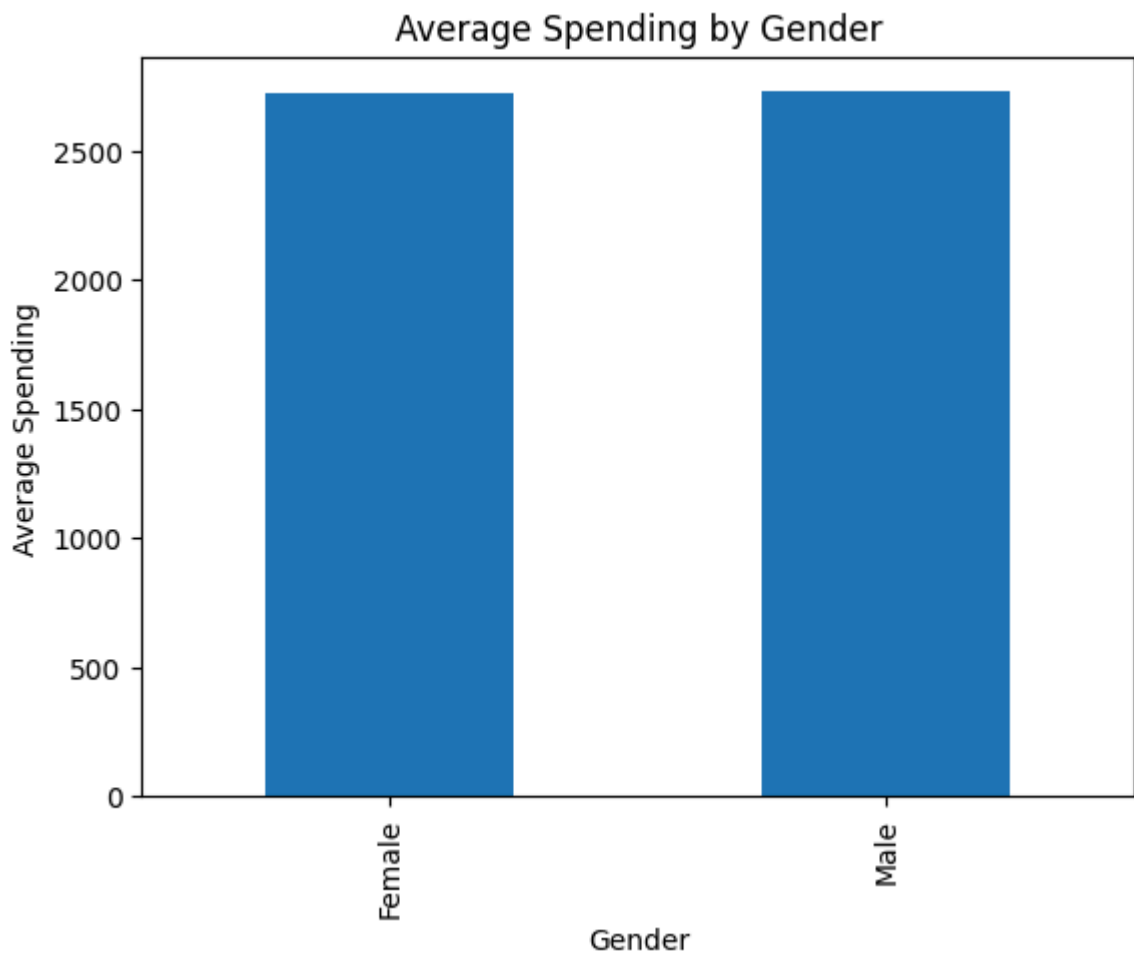
```
In [46]: avg_payment.plot(kind="bar")  
  
plt.title("Average Spending by Payment Method")  
  
plt.xlabel("Payment Method")  
  
plt.ylabel("Average Purchase")  
  
plt.show()
```



```
In [47]: avg_gender = df.groupby("Gender")["Total Purchase Amount"].mean()  
  
avg_gender
```

```
Out[47]: Gender  
Female    2724.104112  
Male      2727.510700  
Name: Total Purchase Amount, dtype: float64
```

```
In [48]: avg_gender.plot(kind="bar")  
  
plt.title("Average Spending by Gender")  
  
plt.xlabel("Gender")  
  
plt.ylabel("Average Spending")  
  
plt.show()
```



In [49]: `df.head()`

Out[49]:

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Age
0	44605	2023-05-03 21:30:02	Home	177	1	2427	PayPal	31
1	44605	2021-05-16 13:57:44	Electronics	174	3	2448	PayPal	31
2	44605	2020-07-13 06:16:57	Books	413	1	2345	Credit Card	31
3	44605	2023-01-17 13:14:36	Electronics	396	3	937	Cash	31
4	44605	2021-05-01 11:29:27	Books	259	4	2598	PayPal	31

EDA Summary

Key findings from the exploratory analysis:

- Product category sales vary significantly.
- Monthly sales reveal seasonal trends.
- Payment methods show different customer preferences.
- Customer ages are spread across multiple age groups.
- Some customers contribute significantly more revenue than others.
- Churned and retained customers can be compared in the next stage using RFM analysis.

EDA Summary

Key findings from the exploratory analysis:

- Product category sales vary significantly.
- Monthly sales reveal seasonal trends.
- Payment methods show different customer preferences.
- Customer ages are spread across multiple age groups.
- Some customers contribute significantly more revenue than others.
- Churned and retained customers can be compared in the next stage using RFM analysis.

```
In [51]: snapshot_date = df["Purchase Date"].max() + pd.Timedelta(days=1)

print(snapshot_date)
```

2023-09-14 18:42:49

```
In [52]: rfm = df.groupby("Customer ID").agg({
    "Purchase Date": lambda x: (snapshot_date - x.max()).days,
    "Customer ID": "count",
    "Total Purchase Amount": "sum"
})

rfm.columns = ["Recency", "Frequency", "Monetary"]

rfm.head()
```

Out[52]:

	Recency	Frequency	Monetary
--	---------	-----------	----------

Customer ID			
1	889	1	5148
2	73	6	16481
3	223	1	4687
4	442	5	7826
5	538	3	7681

```
In [53]: print(rfm.shape)

rfm.describe()
```

(49146, 3)

```
Out[53]:
```

	Recency	Frequency	Monetary
count	49146.000000	49146.000000	49146.000000
mean	311.446588	4.122777	11237.938449
std	281.331150	1.947856	6058.642355
min	1.000000	1.000000	117.000000
25%	95.000000	3.000000	6714.250000
50%	227.000000	4.000000	10524.000000
75%	448.000000	5.000000	14940.750000
max	1352.000000	14.000000	43948.000000

```
In [54]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

rfm_scaled = scaler.fit_transform(rfm)

rfm_scaled
```

```
Out[54]: array([[ 2.05295179, -1.60320309, -1.00517576],
                [-0.84757416,  0.96374785,  0.86539436],
                [-0.31438924, -1.60320309, -1.08126618],
                ...,
                [-1.05729356,  3.01730861,  1.52792576],
                [ 0.16192262,  0.45035766,  0.2428063 ],
                [-0.66984586,  0.96374785,  1.42856256]], shape=(49146, 3))
```

```
In [55]: wcss = []

for i in range(2,11):

    kmeans = KMeans(n_clusters=i,
                    random_state=42)

    kmeans.fit(rfm_scaled)

    wcss.append(kmeans.inertia_)
```

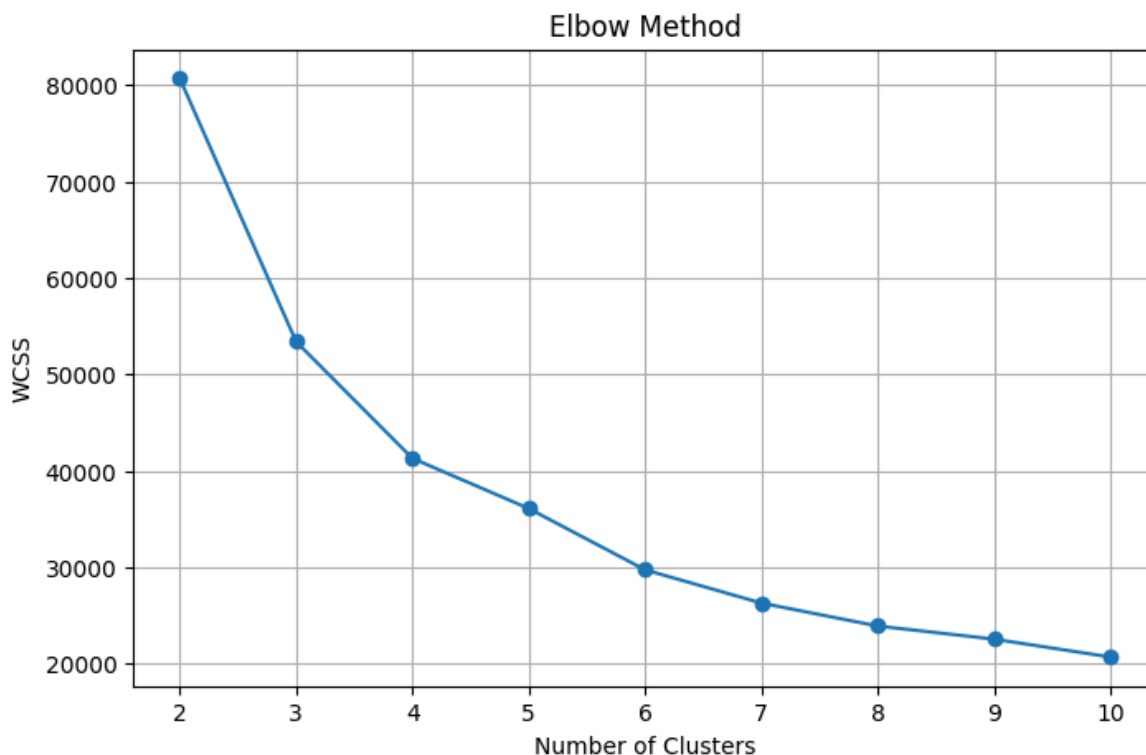
```
In [56]: plt.figure(figsize=(8,5))

plt.plot(range(2,11),
         wcss,
         marker="o")

plt.title("Elbow Method")

plt.xlabel("Number of Clusters")
```

```
plt.ylabel("WCSS")
plt.grid(True)
plt.show()
```



Choose the value of K where the curve starts to bend (the "elbow"). For this project, we will use **4 clusters**.

```
In [57]: kmeans = KMeans(
          n_clusters=4,
          random_state=42
        )

rfm["Cluster"] = kmeans.fit_predict(rfm_scaled)

rfm.head()
```

Out[57]:

	Recency	Frequency	Monetary	Cluster
--	---------	-----------	----------	---------

Customer ID

1	889	1	5148	1
2	73	6	16481	0
3	223	1	4687	2
4	442	5	7826	0
5	538	3	7681	1

```
In [58]: rfm["Cluster"].value_counts()
```

```
Out[58]: Cluster
0      17908
2      15118
1       8447
3       7673
Name: count, dtype: int64
```

```
In [59]: cluster_summary = rfm.groupby("Cluster").mean()

cluster_summary
```

```
Out[59]:
```

	Recency	Frequency	Monetary
Cluster			
0	216.272057	4.817735	13213.982075
1	805.883391	2.304132	6110.569551
2	226.515081	2.721326	6729.272457
3	156.601329	7.264173	21153.995830

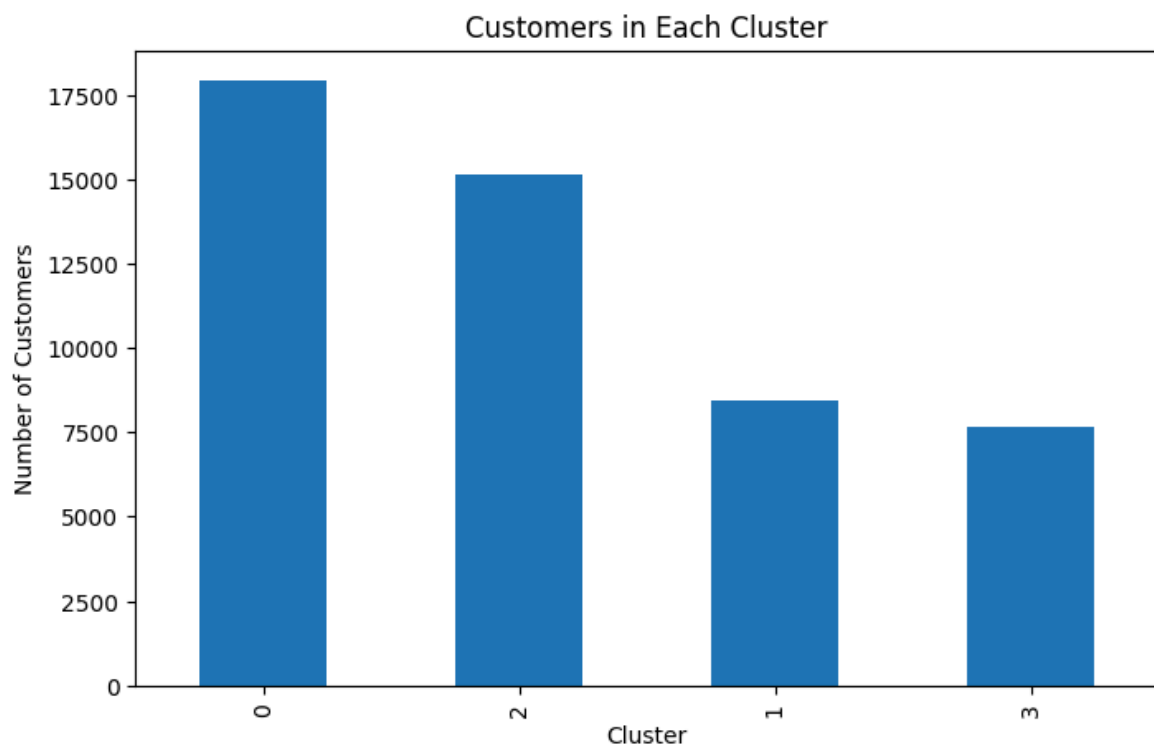
```
In [60]: rfm["Cluster"].value_counts().plot(
        kind="bar",
        figsize=(8,5)
    )

plt.title("Customers in Each Cluster")

plt.xlabel("Cluster")

plt.ylabel("Number of Customers")

plt.show()
```



```
In [61]: plt.figure(figsize=(8,6))

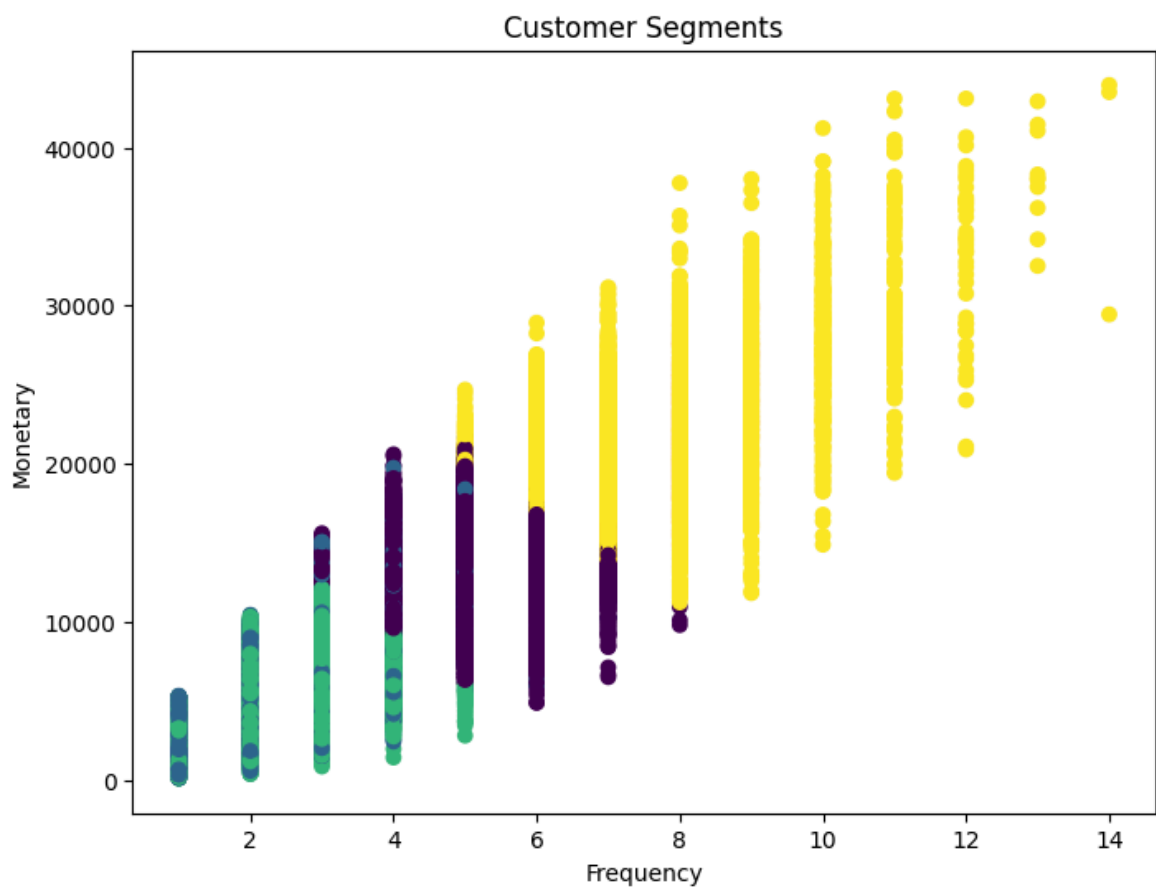
plt.scatter(
    rfm["Frequency"],
    rfm["Monetary"],
    c=rfm["Cluster"]
)

plt.xlabel("Frequency")

plt.ylabel("Monetary")

plt.title("Customer Segments")

plt.show()
```



```
In [62]: plt.figure(figsize=(8,6))

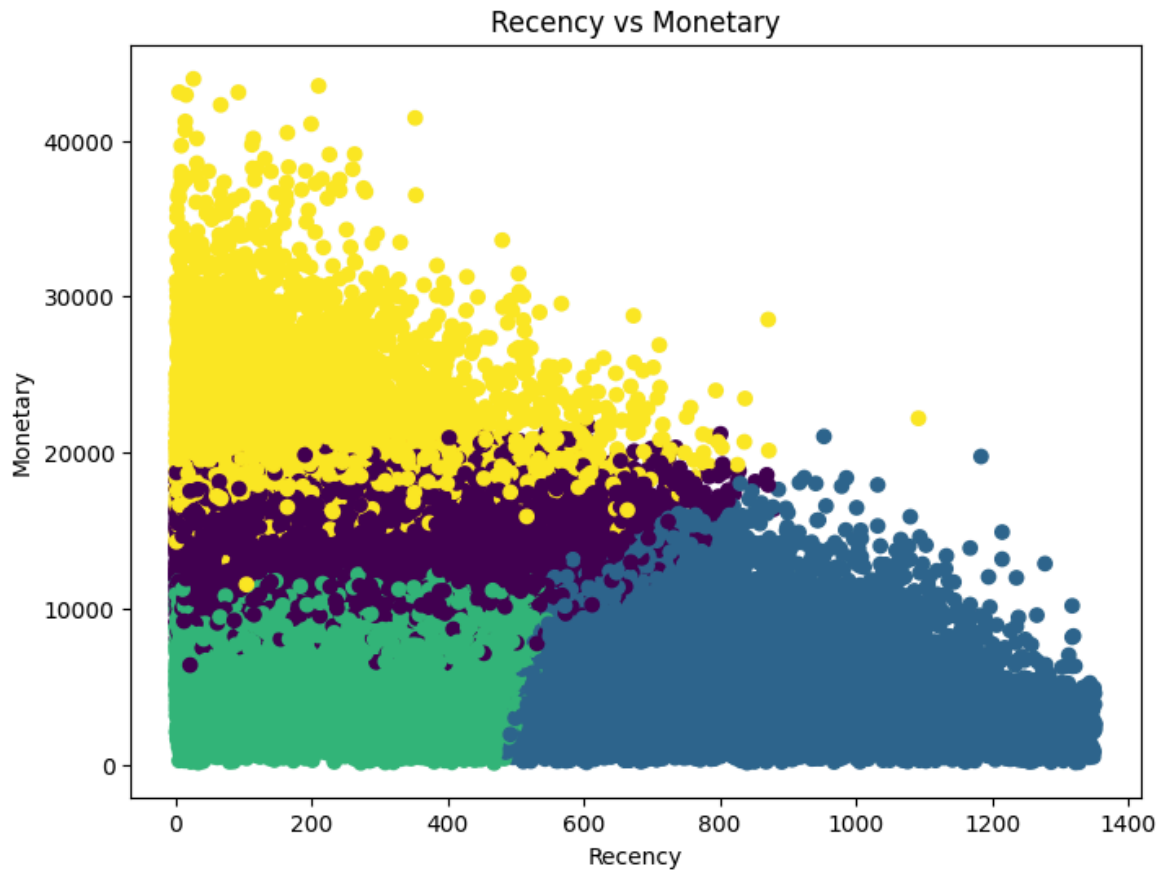
plt.scatter(
    rfm["Recency"],
    rfm["Monetary"],
    c=rfm["Cluster"]
)

plt.xlabel("Recency")

plt.ylabel("Monetary")

plt.title("Recency vs Monetary")

plt.show()
```



```
In [63]: plt.figure(figsize=(8,6))

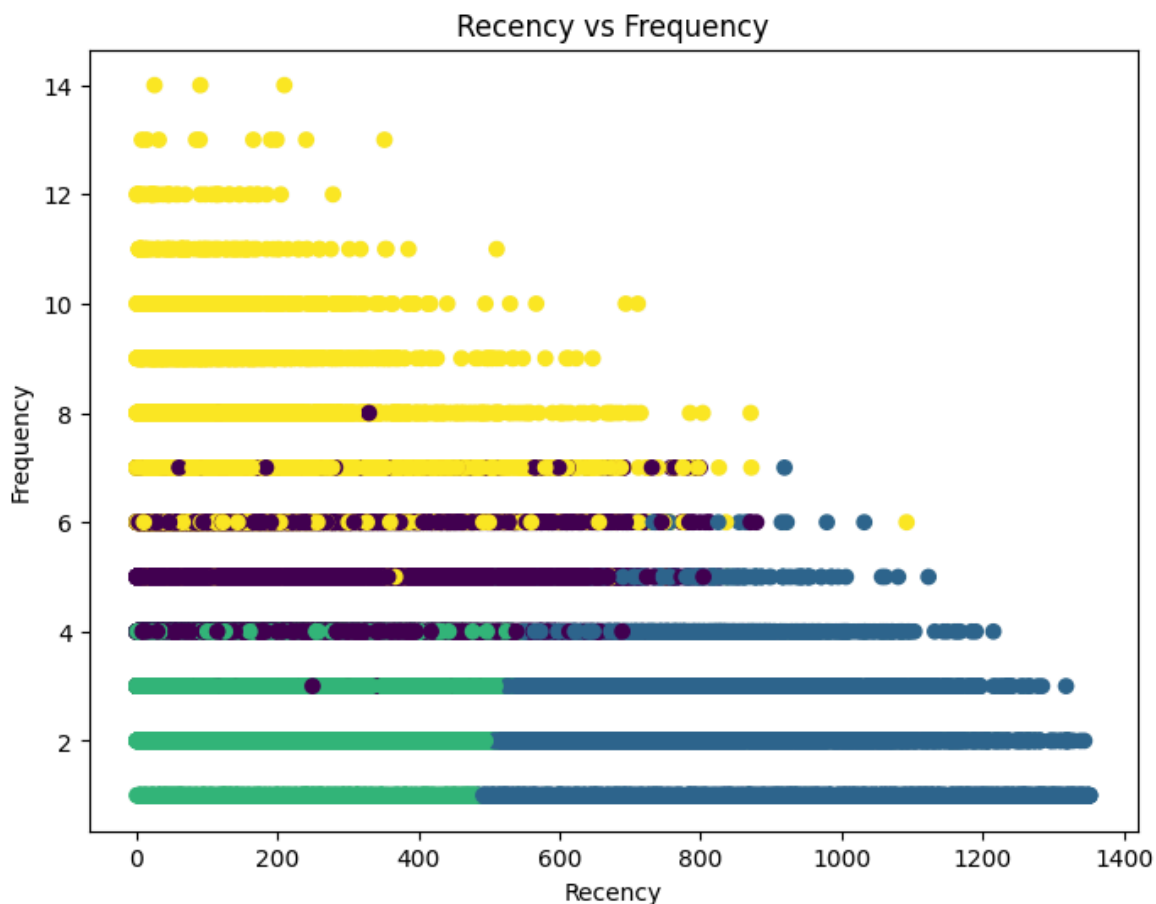
plt.scatter(
    rfm["Recency"],
    rfm["Frequency"],
    c=rfm["Cluster"]
)

plt.xlabel("Recency")

plt.ylabel("Frequency")

plt.title("Recency vs Frequency")

plt.show()
```



In [64]: `rfm.to_csv("Customer_Segments.csv")`

In [65]: `rfm.head()`

Out[65]:

	Recency	Frequency	Monetary	Cluster
--	---------	-----------	----------	---------

Customer ID

1	889	1	5148	1
2	73	6	16481	0
3	223	1	4687	2
4	442	5	7826	0
5	538	3	7681	1

Customer Segment Profiles

Cluster 0

- High purchase frequency
- High total spending
- Loyal customers

Cluster 1

- Recent customers
- Moderate spending
- Potential loyal customers

Cluster 2

- Low purchase frequency
- Low spending
- New or inactive customers

Cluster 3

- High recency (long time since last purchase)
- Churn risk customers

```
In [66]: cluster_summary.round(2)
```

```
Out[66]:
```

	Recency	Frequency	Monetary
Cluster			
0	216.27	4.82	13213.98
1	805.88	2.30	6110.57
2	226.52	2.72	6729.27
3	156.60	7.26	21154.00

Cluster			
0	216.27	4.82	13213.98
1	805.88	2.30	6110.57
2	226.52	2.72	6729.27
3	156.60	7.26	21154.00

Purchase Patterns, Retention Trends and Churn Analysis

This section analyzes customer purchasing behavior, customer retention, and churn risk to provide business insights.

```
In [67]: purchase_pattern = df.groupby("Product Category").agg({
    "Quantity": "sum",
    "Total Purchase Amount": "sum"
})

purchase_pattern
```

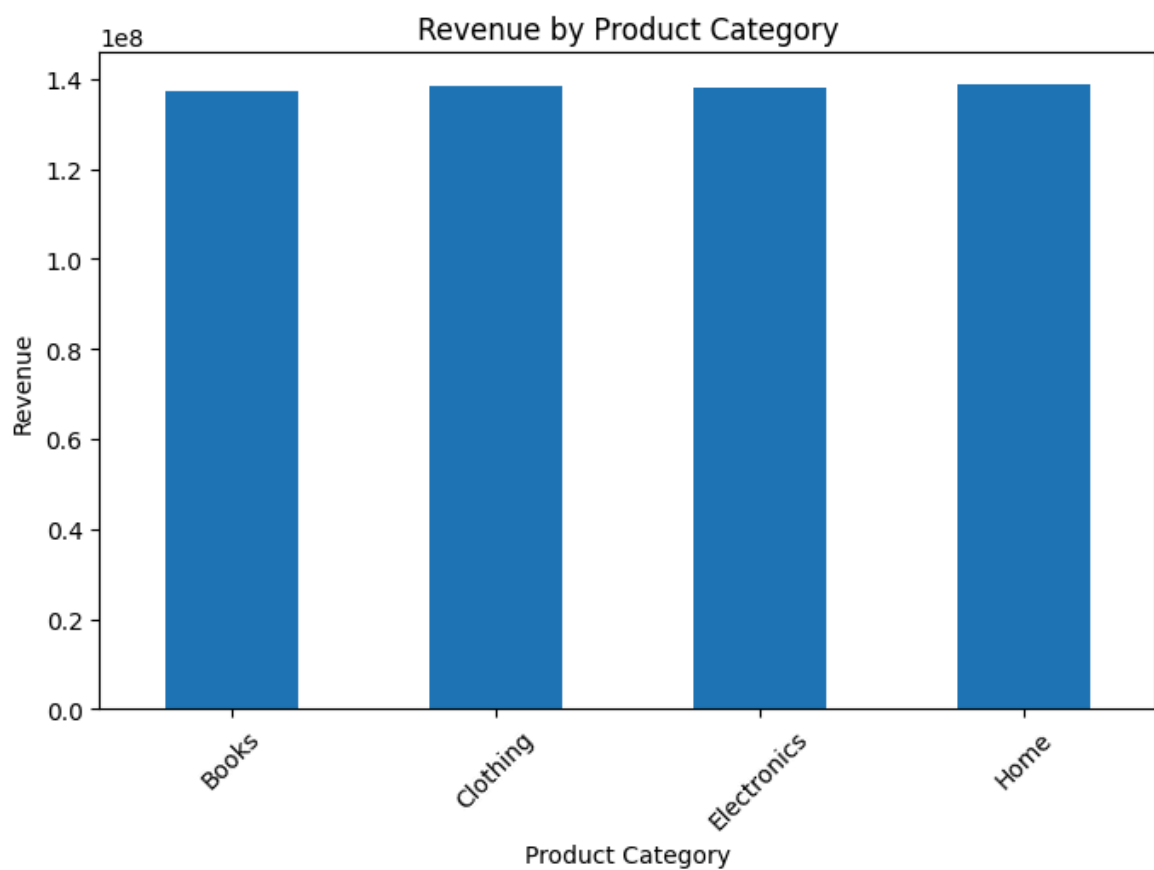

Out[67]:

	Quantity	Total Purchase Amount
Product Category		
Books	151332	137145086
Clothing	153372	138363540
Electronics	152111	137884886
Home	152745	138906211

```
In [68]: purchase_pattern["Total Purchase Amount"].plot(
        kind="bar",
        figsize=(8,5)
    )

plt.title("Revenue by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Revenue")
plt.xticks(rotation=45)

plt.show()
```



```
In [69]: monthly_purchase = df.groupby("Month Name")["Total Purchase Amount"].sum()

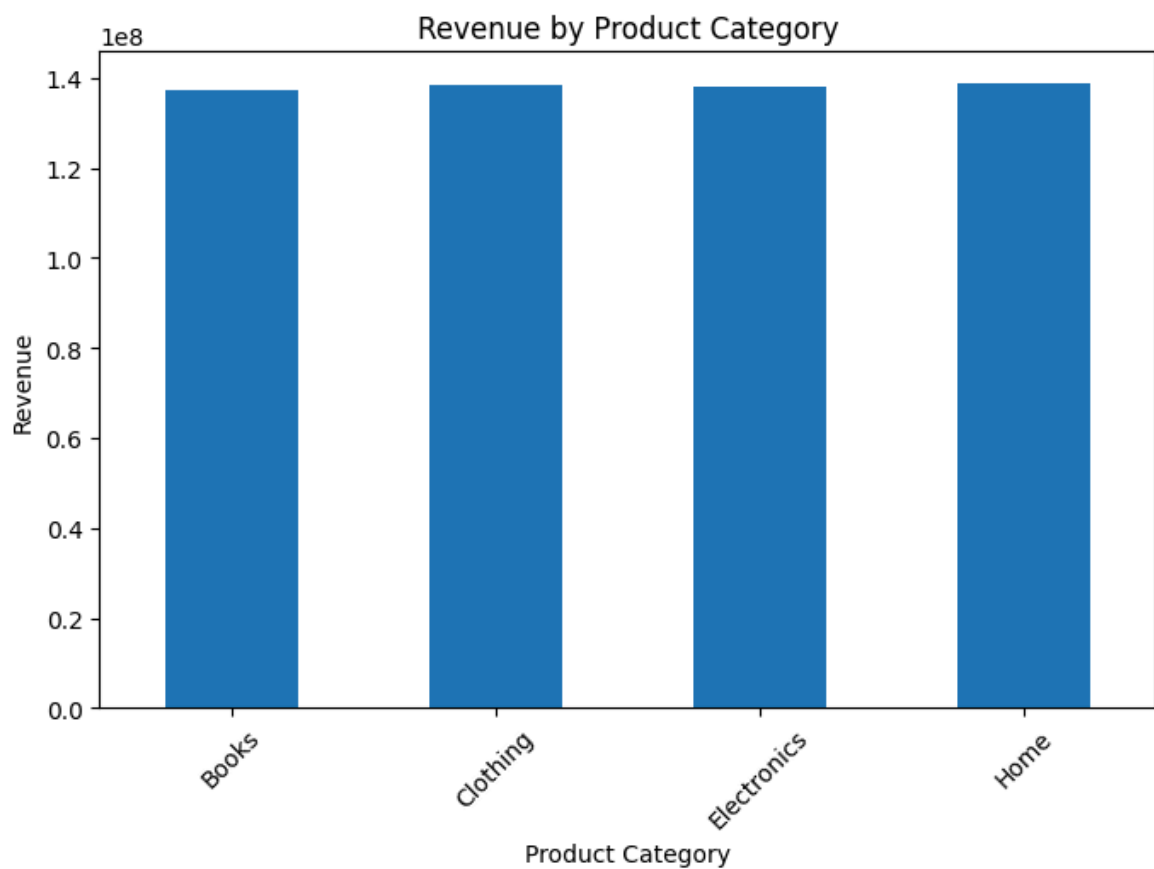
monthly_purchase
```

```
Out[69]: Month Name
April      48996901
August     50991460
December   38372480
February   45980491
January    50295819
July       50436195
June       48934737
March      50417750
May        50955398
November   37094782
October    38214132
September  41609578
Name: Total Purchase Amount, dtype: int64
```

```
In [70]: purchase_pattern["Total Purchase Amount"].plot(
        kind="bar",
        figsize=(8,5)
    )

plt.title("Revenue by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Revenue")
plt.xticks(rotation=45)

plt.show()
```



```
In [71]: monthly_purchase = df.groupby("Month Name")["Total Purchase Amount"].sum()

monthly_purchase
```

```
Out[71]: Month Name
April      48996901
August     50991460
December   38372480
February   45980491
January    50295819
July       50436195
June       48934737
March      50417750
May        50955398
November   37094782
October    38214132
September  41609578
Name: Total Purchase Amount, dtype: int64
```

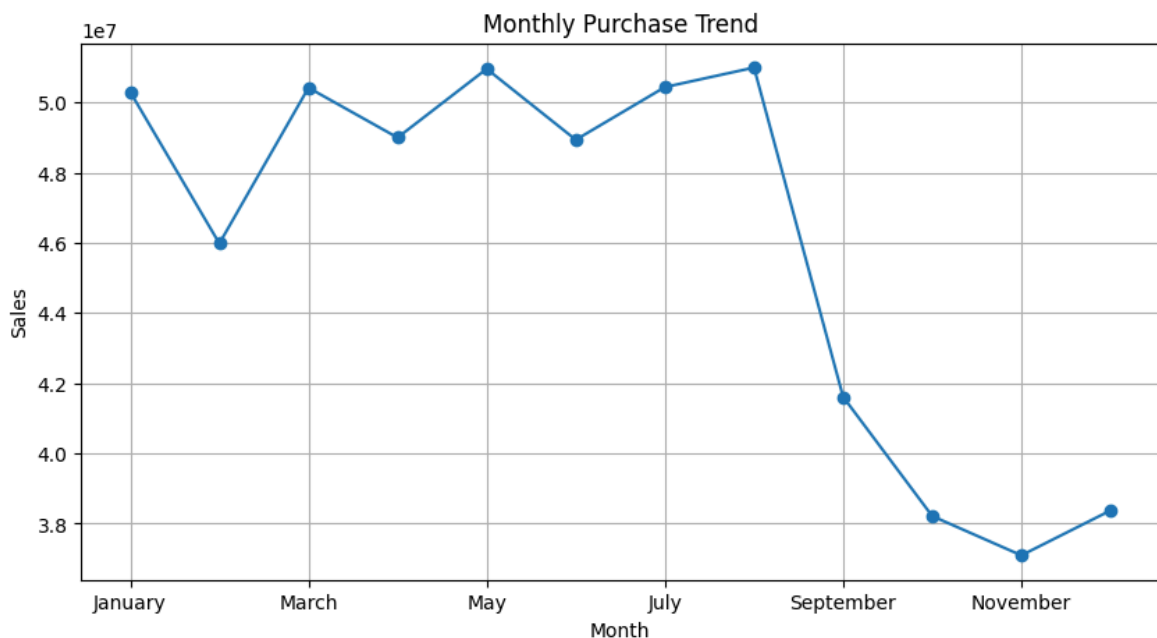
```
In [72]: month_order = ["January", "February", "March", "April", "May", "June",
                        "July", "August", "September", "October", "November", "December"]

monthly_purchase = monthly_purchase.reindex(month_order)

monthly_purchase.plot(
    kind="line",
    marker="o",
    figsize=(10,5)
)

plt.title("Monthly Purchase Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)

plt.show()
```



```
In [73]: retention = df.groupby("Month Name")["Customer ID"].nunique()

retention = retention.reindex(month_order)

retention
```

```
Out[73]: Month Name
January    15453
February   14319
March      15559
April      15041
May        15606
June       15042
July       15518
August     15663
September  13255
October    12148
November   11902
December   12261
Name: Customer ID, dtype: int64
```

```
In [74]: retention.plot(
        marker="o",
        figsize=(10,5)
    )

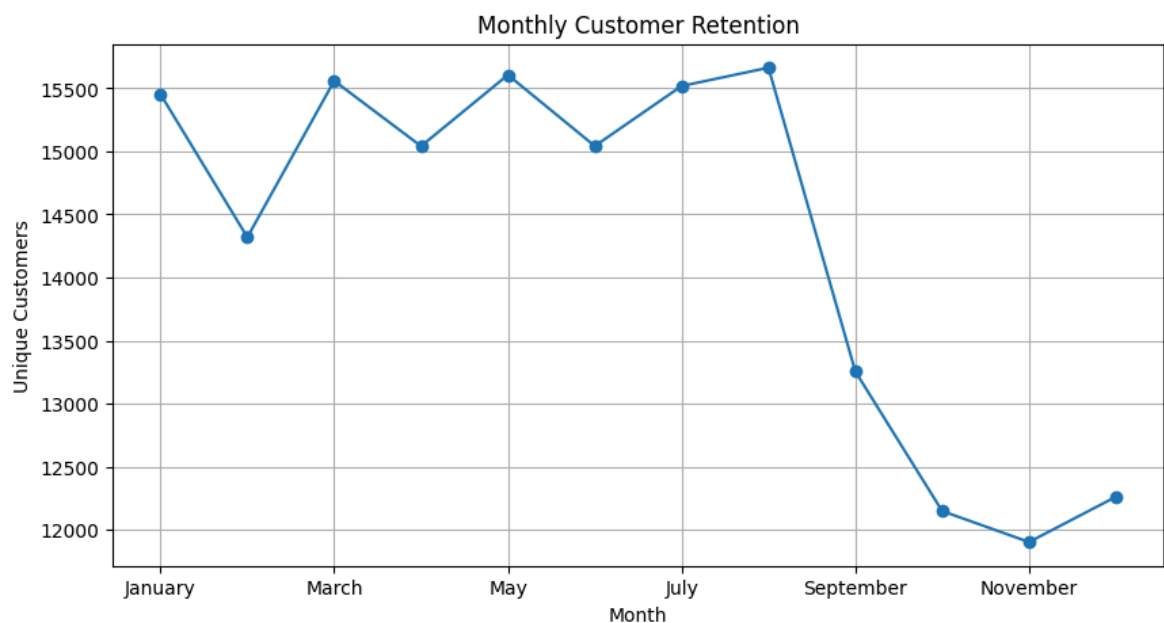
plt.title("Monthly Customer Retention")

plt.xlabel("Month")

plt.ylabel("Unique Customers")

plt.grid(True)

plt.show()
```

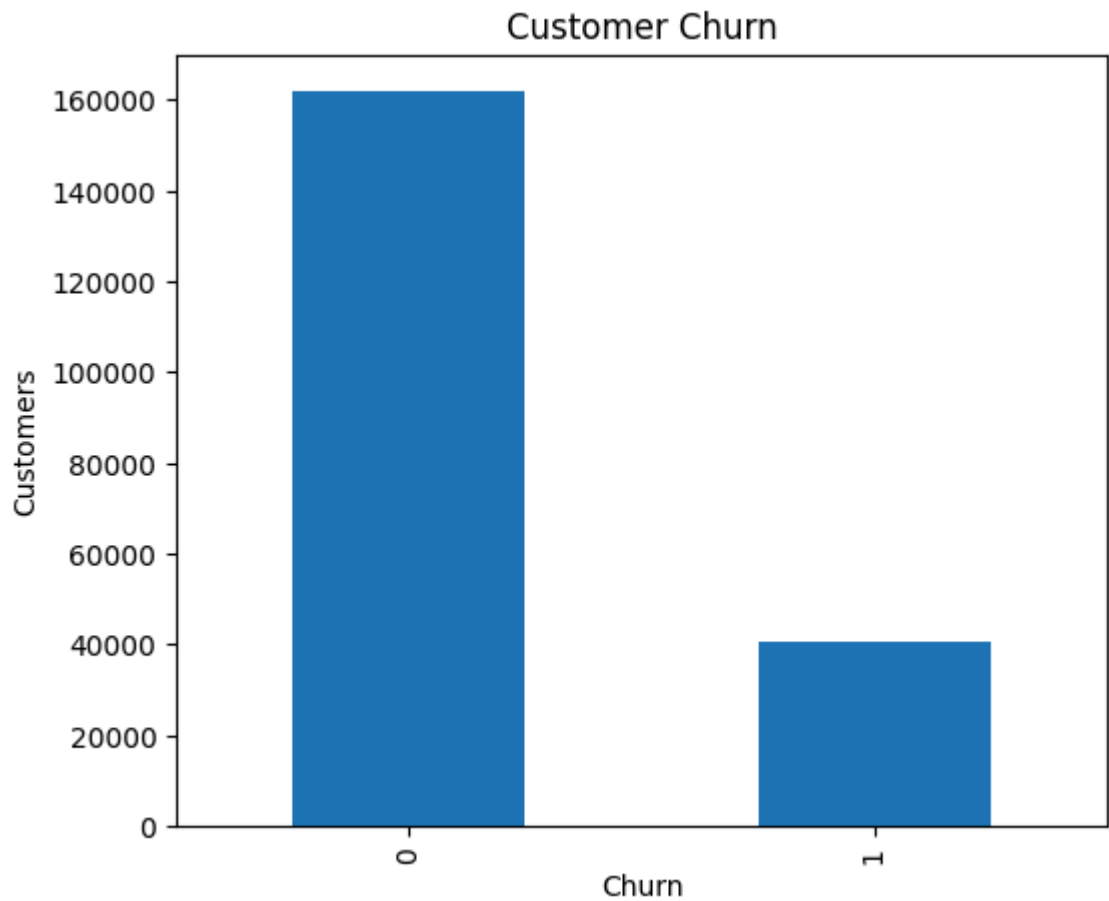


```
In [75]: df["Churn"].value_counts()
```

```
Out[75]: Churn
0      161874
1       40744
Name: count, dtype: int64
```

```
In [76]: df["Churn"].value_counts().plot(
        kind="bar",
        figsize=(6,5)
    )
```

```
)  
plt.title("Customer Churn")  
plt.xlabel("Churn")  
plt.ylabel("Customers")  
plt.show()
```



```
In [77]: churn_rate = df["Churn"].mean()*100  
print("Customer Churn Rate:", round(churn_rate,2), "%")
```

Customer Churn Rate: 20.11 %

```
In [78]: at_risk = rfm[rfm["Recency"]>90]  
at_risk.head()
```

Out[78]:

	Recency	Frequency	Monetary	Cluster
Customer ID				
1	889	1	5148	1
3	223	1	4687	2
4	442	5	7826	0
5	538	3	7681	1
6	141	8	24812	3

In [79]: `print("Customers at Risk:", len(at_risk))`

Customers at Risk: 37331

In [80]: `rfm.groupby("Cluster")["Monetary"].mean()`

Out[80]: Cluster
0 13213.982075
1 6110.569551
2 6729.272457
3 21153.995830
Name: Monetary, dtype: float64

In [81]: `rfm.groupby("Cluster")["Monetary"].mean().plot(
 kind="bar",
 figsize=(7,5)
)

plt.title("Average Spending by Cluster")

plt.xlabel("Cluster")

plt.ylabel("Average Spending")

plt.show()`



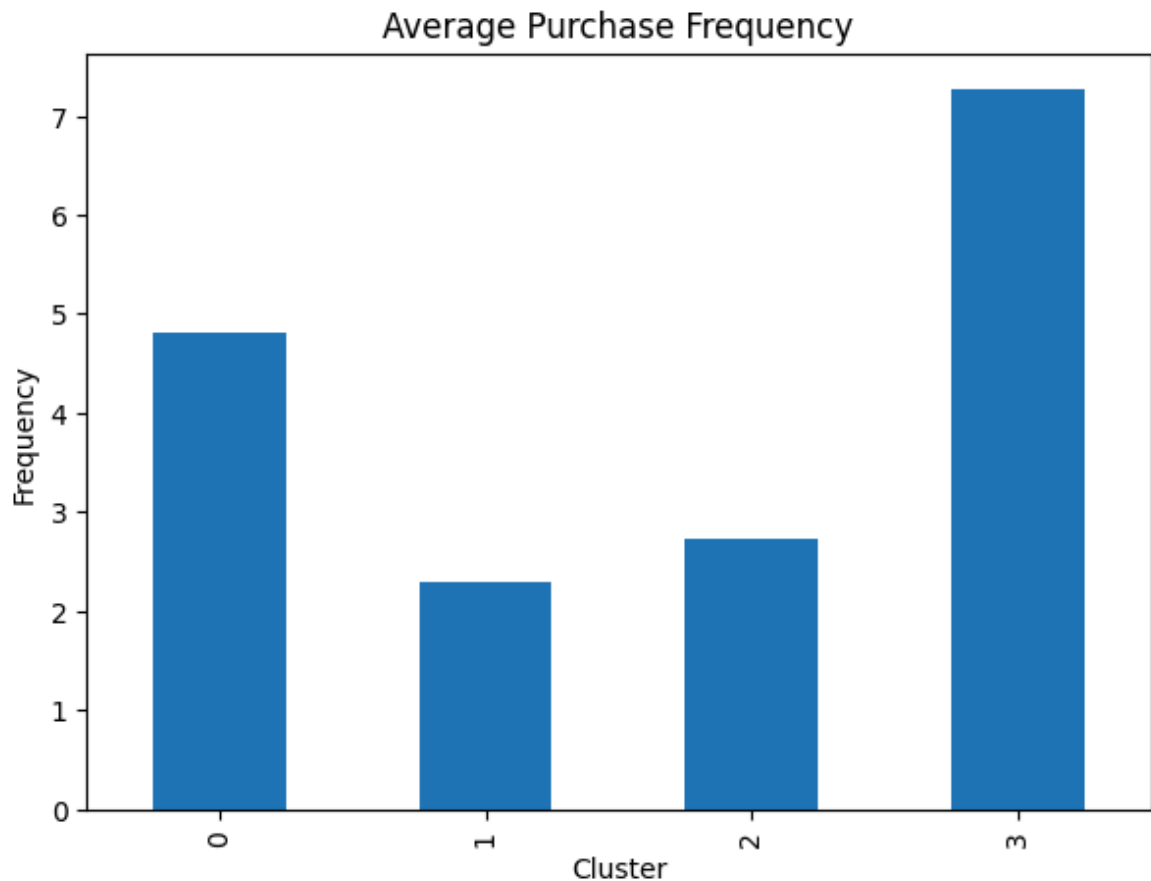
```
In [82]: rfm.groupby("Cluster")["Frequency"].mean().plot(
          kind="bar",
          figsize=(7,5)
        )

plt.title("Average Purchase Frequency")

plt.xlabel("Cluster")

plt.ylabel("Frequency")

plt.show()
```



```
In [83]: rfm.to_csv("Customer_Segments.csv")
```

```
In [84]: rfm.head()
```

```
Out[84]:
```

	Recency	Frequency	Monetary	Cluster
--	---------	-----------	----------	---------

Customer ID				
1	889	1	5148	1
2	73	6	16481	0
3	223	1	4687	2
4	442	5	7826	0
5	538	3	7681	1

Business Insights

Key Findings

- Product categories contribute differently to total revenue.
- Customer purchases vary across months, indicating seasonal demand.
- K-Means clustering successfully grouped customers based on purchasing behavior.
- A small group of customers contributes significantly higher revenue.
- Customers with high Recency values are more likely to churn.

Recommendations for Alfido Tech

1. Launch personalized email or SMS campaigns targeting customers with high Recency values to reduce churn.
2. Introduce a loyalty rewards program for high-value customers to encourage repeat purchases.
3. Promote top-selling product categories using personalized recommendations and bundled offers.
4. Run promotional campaigns during months with lower sales to improve customer engagement and retention.
5. Offer exclusive discounts or early access to premium customers based on their RFM segment to increase customer lifetime value.

Conclusion

This project analyzed customer transactions using data cleaning, feature engineering, exploratory data analysis, RFM analysis, and K-Means clustering.

Customer segments were identified based on Recency, Frequency, and Monetary values. The analysis highlighted purchasing patterns, retention trends, and churn risks.

The findings can help Alfido Tech improve customer engagement, increase retention, and design targeted marketing campaigns.

In []: